

Обучение нейронной сети. Метод обратного распространения ошибки

ЛЕКЦИЯ №3

1. Алгоритм обратного распространения ошибки.
2. Градиентные методы.
3. Универсальные аппроксимационные свойства Персептрона.
4. Много-классовая классификация
5. Примеры

- 1. Алгоритм обратного распространения ошибки.**
- 2. Градиентные методы.**
- 3. Универсальные аппроксимационные свойства
Персептрона.**
- 4. Много-классовая классификация**
- 5. Примеры**

Многослойный персептрон

$$y_j = f_j \left(\sum_{i=0,n} w_{ij} f_i \left(\sum_{k=0,m} w_{ki} x_k \right) \right)$$

m – количество нейронов в слое l

n – количество нейронов в слое $l-1$

$$W^{(l)} = \begin{bmatrix} w_{11}^{(l)} & \dots & w_{1m}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{n1}^{(l)} & \dots & w_{nm}^{(l)} \end{bmatrix}$$

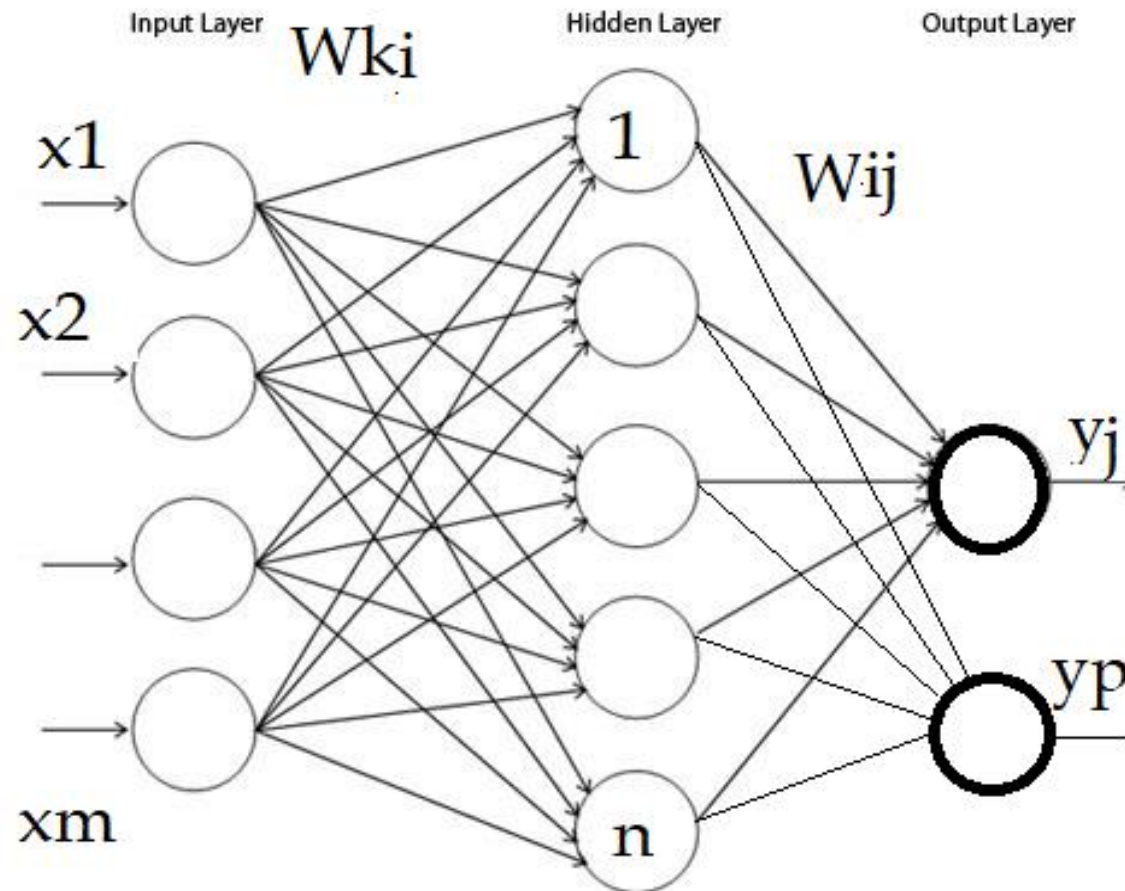
$$\mathbf{b}^{(l)} = [b_1^{(l)} \quad \dots \quad b_m^{(l)}]^\top \quad \mathbf{z}^{(l)} = [z_1^{(l)} \quad \dots \quad z_m^{(l)}]^\top$$

$$\mathbf{a}^{(l-1)} = [a_1^{(l-1)} \quad \dots \quad a_n^{(l-1)}]^\top \quad \mathbf{a}^{(l)} = [a_1^{(l)} \quad \dots \quad a_m^{(l)}]^\top$$

$$\sigma(\mathbf{z}^{(l)}) = [\sigma(z_1^{(l)}) \quad \dots \quad \sigma(z_m^{(l)})]^\top$$

$$\mathbf{z}^{(l)} = W^{(l)\top} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$



Алгоритм обратного распространения ошибки (backpropagation)

$$D : \{x_i, y_i, i=1, n\}, \quad y_j = f_j \left(\sum_{i=0, n} w_{ij} f_i \left(\sum_{k=0, m} w_{ki} x_k \right) \right)$$

W_{ji} – синаптические веса нейронов в j -й момент времени.

$$Y = X^T W$$

$e_i = y_i - \tilde{y}_i$ – сигнал ошибки

Функционал ошибок

$$E = \frac{1}{2} (Y - \tilde{Y})^2$$

Задача – выбор вектора W , минимизирующего ошибку E

Алгоритм обучения однослойного персептрона

$$\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, \dots, \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$$

1. Initialize $\mathbf{w} := 0^m$
2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w})$
 - (b) $err := (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$

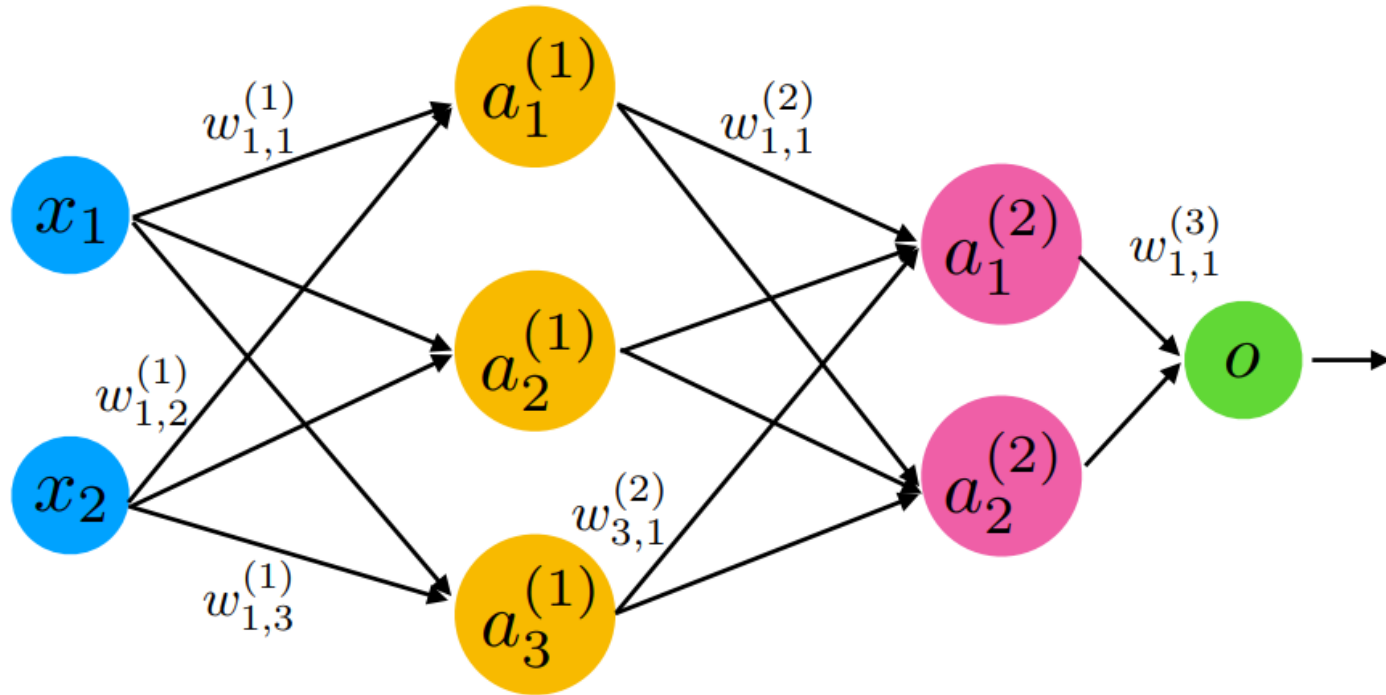
Алгоритм обратного распространения ошибки

L – количество слоев нейронной сети

$\mathbf{a}^{(L)} = \boldsymbol{\alpha}(\mathbf{x})$ – выходной сигнал нейронной сети

$\mathcal{C} = \mathcal{C}_{(x,y)} = \frac{\|\mathbf{a}^{(L)} - \mathbf{y}\|^2}{2}$ – ошибка на прецеденте (x, y)

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \begin{cases} 0, & \mathbf{w}^T \mathbf{x} + b \leq 0 \\ 1, & \mathbf{w}^T \mathbf{x} + b > 0 \end{cases}$$



$$\mathcal{C}(y, o) = l \quad z_i^{(l)} = b_i^{(l)} + \sum_{j=1}^n w_{ji}^{(l)} a_j^{(l-1)}$$

$$a_i^{(l)} = \sigma(z_i^{(l)})$$

Алгоритм обратного распространения ошибки

$\mathbf{a}^{(L)} = \boldsymbol{\alpha}(\mathbf{x})$ – выходной сигнал нейронной сети

$\mathcal{C} = \mathcal{C}_{(x,y)} = \frac{\|\mathbf{a}^{(L)} - \mathbf{y}\|^2}{2}$ – ошибка на прецеденте (x, y)

$$\mathbb{C} = \frac{1}{|V|} \sum_{(x,y) \in V} \mathcal{C}_{(x,y)}$$

$$\begin{aligned}\nabla_{\mathbf{w}} \mathbb{C} &= \frac{1}{|V|} \sum_{(x,y) \in V} \nabla_{\mathbf{w}} \mathcal{C}_{(x,y)} \\ \nabla_{\mathbf{b}} \mathbb{C} &= \frac{1}{|V|} \sum_{(x,y) \in V} \nabla_{\mathbf{b}} \mathcal{C}_{(x,y)}\end{aligned}$$

$$z_i^{(l)} = b_i^{(l)} + \sum_{j=1}^n w_{ji}^{(l)} a_j^{(l-1)}$$

$$a_i^{(l)} = \sigma(z_i^{(l)})$$

$$\nabla_{\mathbf{w}} \mathbb{C} = \left(\frac{\partial \mathbb{C}}{\partial w_1}, \dots, \frac{\partial \mathbb{C}}{\partial w_Q} \right)$$

$$\nabla_{\mathbf{b}} \mathbb{C} = \left(\frac{\partial \mathbb{C}}{\partial b_1}, \dots, \frac{\partial \mathbb{C}}{\partial b_P} \right)$$

Алгоритм обратного распространения ошибки

Шаг 1. Взять образец (x, y) и подать сигнал x на вход нейронной сети

Шаг 2. Последовательно вычислить выходные сигналы $a^{(l)}$ для каждого слоя

Шаг 3. Вычислить меру влияния $\delta^{(L)}$ нейронов выходного слоя L на ошибку C

Шаг 4. Вычислить в обратном порядке меру влияния $\delta^{(l)}$ на ошибку
для каждого слоя

Шаг 5. Используя $\delta^{(l)}$, вычислить $\nabla_w C_{(x,y)}$ и $\nabla_b C_{(x,y)}$ для
каждого слоя l

$$\nabla_w C = \left(\frac{\partial C}{\partial w_1}, \dots, \frac{\partial C}{\partial w_Q} \right)$$

Шаг 6. Выполнив шаги 1-5 для всей подвыборки V_i , вычислить

$$\nabla_b C = \left(\frac{\partial C}{\partial b_1}, \dots, \frac{\partial C}{\partial b_P} \right)$$

$$\nabla_w C_{V_i} := \frac{1}{|V_i|} \sum_{(x,y) \in V_i} \nabla_w C_{(x,y)}$$

$$\nabla_b C_{V_i} := \frac{1}{|V_i|} \sum_{(x,y) \in V_i} \nabla_b C_{(x,y)}$$

Алгоритм обратного распространения ошибки

$$z_i^{(l)} = b_i^{(l)} + \sum_{j=1}^n w_{ji}^{(l)} a_j^{(l-1)}$$
$$a_i^{(l)} = \sigma(z_i^{(l)})$$

$$\delta_i^{(L)} = \frac{\partial C}{\partial z_i^{(L)}}$$

$$= \frac{\partial C}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}}$$

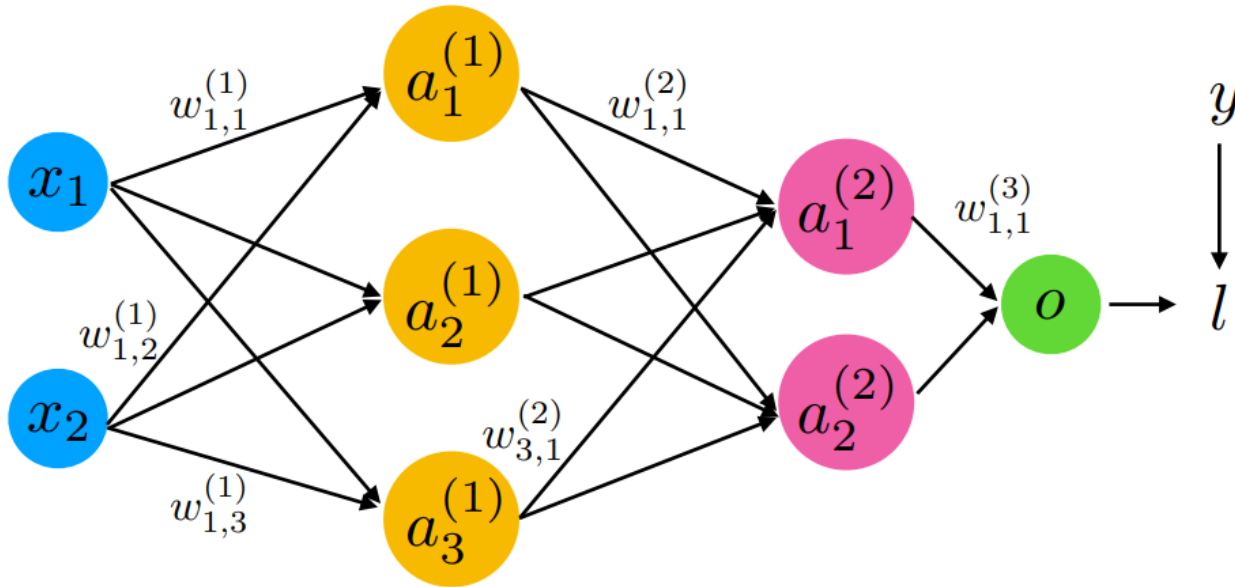
$$= \frac{\partial C}{\partial a_i^{(L)}} \cdot \sigma'(z_i^{(L)})$$

$$\delta_i^{(l)} ::= \frac{\partial C}{\partial z_i^{(l)}} \text{ // для одного нейрона}$$
$$\boldsymbol{\delta}^{(l)} ::= \nabla_{\mathbf{z}^{(l)}} C \text{ // для слоя}$$

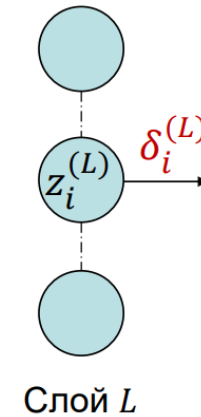
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

Алгоритм обратного распространения ошибки

Влияние нейрона на величину ошибки C



$$\delta_i^{(L)} = \frac{\partial C}{\partial a_i^{(L)}} \cdot \sigma' \left(z_i^{(L)} \right)$$



Алгоритм обратного распространения ошибки

Влияние нейрона на величину ошибки C

$$\delta_i^{(L)} = \frac{\partial C}{\partial a_i^{(L)}} \cdot \sigma' \left(z_i^{(L)} \right)$$

$$\delta_i^{(L)} = (a_i^{(L)} - y_i) \cdot \sigma' \left(z_i^{(L)} \right)$$

Лемма 1: $\frac{\partial C}{\partial a_i^{(L)}} = a_i^{(L)} - y_i$

$$C = \frac{\|a^{(L)} - y\|^2}{2} = \frac{\sum_j (a_j^{(L)} - y_j)^2}{2} = \frac{\sum_j (a_j^{(L)^2} - 2a_j^{(L)}y_j + y_j^2)}{2}$$

$\Downarrow$

$$\frac{\partial C}{\partial a_i^{(L)}} = \frac{2a_i^{(L)} - 2y_i}{2} = a_i^{(L)} - y_i$$

Алгоритм обратного распространения ошибки

Лемма 2: $\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} = w_{ji}^{(l+1)} \sigma' (z_j^{(l)})$

$$z_i^{(l+1)} = b_i^{(l+1)} + \sum_j w_{ji}^{(l+1)} \sigma (z_j^{(l)})$$



$$\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} = w_{ji}^{(l+1)} \sigma' (z_j^{(l)})$$

Алгоритм обратного распространения ошибки

Лемма 3: $\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$

$$z_i^{(l)} = b_i^{(l)} + \sum_j w_{ji}^{(l)} a_j^{(l-1)}$$

$\Downarrow$

$$\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$$

Алгоритм обратного распространения ошибки

Лемма 4: $\frac{\partial z_i^{(l)}}{\partial w_{ji}^{(l)}} = a_j^{(l-1)}$

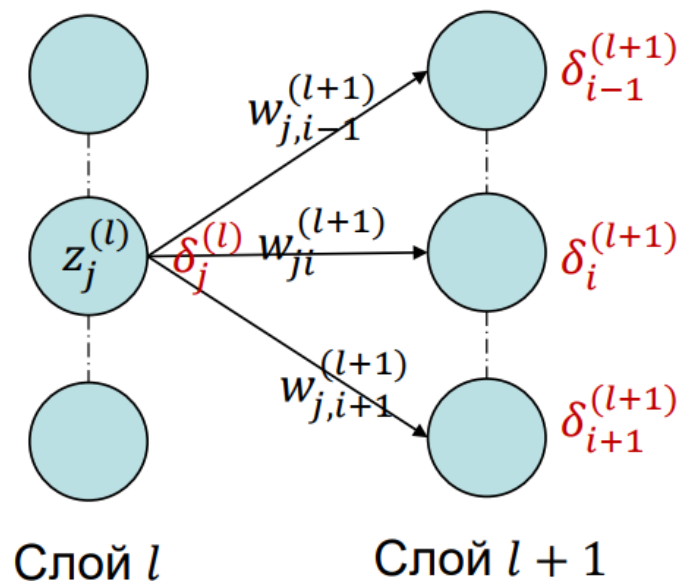
$$z_i^{(l)} = b_i^{(l)} + \sum_j w_{ji}^{(l)} a_j^{(l-1)}$$

$\Downarrow$

$$\frac{\partial z_i^{(l)}}{\partial w_{ji}^{(l)}} = a_j^{(l-1)}$$

Влияние нейронов слоя на величину ошибки S

$$\delta_j^{(l)} = \sigma' \left(z_j^{(l)} \right) \sum_i w_{ji}^{(l+1)} \delta_i^{(l+1)}$$



Алгоритм обратного распространения ошибки

Влияние слоя

$$\delta_j^{(l)} = \frac{\partial C}{\partial z_j^{(l)}}$$

$$\begin{aligned} \delta_i^{(l)} &::= \frac{\partial C}{\partial z_i^{(l)}} // \text{ для одного нейрона} \\ \delta^{(l)} &::= \nabla_{\mathbf{z}^{(l)}} C // \text{ для слоя} \end{aligned}$$

$$\delta_j^{(l)} = \sigma' \left(z_j^{(l)} \right) \sum_i w_{ji}^{(l+1)} \delta_i^{(l+1)}$$

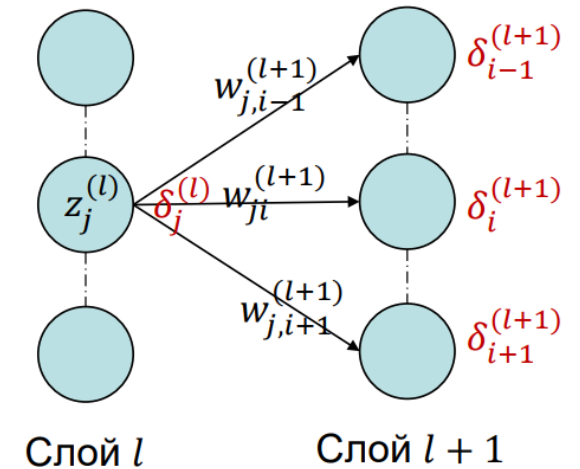
$$\begin{aligned} &= \sum_i \frac{\partial C}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} \\ &= \sum_i \delta_i^{(l+1)} w_{ji}^{(l+1)} \sigma' \left(z_j^{(l)} \right) \\ &= \sigma' \left(z_j^{(l)} \right) \sum_i \delta_i^{(l+1)} w_{ji}^{(l+1)} \end{aligned}$$

Лемма 2: $\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} = w_{ji}^{(l+1)} \sigma' \left(z_j^{(l)} \right)$

$$z_i^{(l+1)} = b_i^{(l+1)} + \sum_j w_{ji}^{(l+1)} \sigma \left(z_j^{(l)} \right)$$

⇓

$$\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} = w_{ji}^{(l+1)} \sigma' \left(z_j^{(l)} \right)$$



Алгоритм обратного распространения ошибки

Влияние смещения

$$\frac{\partial C}{\partial b_i^{(l)}} = \delta_i^{(l)}$$

$$\frac{\partial C}{\partial b_i^{(l)}} = \frac{\partial C}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}}$$

$$\delta_i^{(l)} ::= \frac{\partial C}{\partial z_i^{(l)}} \text{ // для одного нейрона}$$
$$\delta^{(l)} ::= \nabla_{\mathbf{z}^{(l)}} C \text{ // для слоя}$$

$$= \delta_i^{(l)} \cdot \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}}$$

$$= \delta_i^{(l)}$$

Лемма 3: $\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$

$$z_i^{(l)} = b_i^{(l)} + \sum_j w_{ji}^{(l)} a_j^{(l-1)}$$

$\Downarrow$

$$\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$$

Алгоритм обратного распространения ошибки

$$\frac{\partial C}{\partial w_{ji}^{(l)}} = \delta_i^{(l)} \cdot a_j^{(l-1)}$$

$$\begin{aligned}\frac{\partial C}{\partial w_{ij}^{(l)}} &= \frac{\partial C}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} \\ &= \delta_i^{(l)} \cdot \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} \\ &= \delta_i^{(l)} \cdot a_j^{(l-1)}\end{aligned}$$

$\delta_i^{(l)} ::= \frac{\partial C}{\partial z_i^{(l)}} //$ для одного нейрона
 $\delta^{(l)} ::= \nabla_{\mathbf{z}^{(l)}} C //$ для слоя

Лемма 4: $\frac{\partial z_i^{(l)}}{\partial w_{ji}^{(l)}} = a_j^{(l-1)}$

$$z_i^{(l)} = b_i^{(l)} + \sum_j w_{ji}^{(l)} a_j^{(l-1)}$$

$\Downarrow$

$$\frac{\partial z_i^{(l)}}{\partial w_{ji}^{(l)}} = a_j^{(l-1)}$$

Алгоритм обратного распространения ошибки

Формулы для вычисления алгоритма

$$\delta_i^{(L)} = (a_i^{(L)} - y_i) \cdot \sigma' \left(z_i^{(L)} \right)$$

$$\delta_j^{(l)} = \sigma' \left(z_j^{(l)} \right) \sum_i w_{ji}^{(l+1)} \delta_i^{(l+1)}$$

$$\frac{\partial C}{\partial b_i^{(l)}} = \delta_i^{(l)}$$

$$\frac{\partial C}{\partial w_{ji}^{(l)}} = \delta_i^{(l)} \cdot a_j^{(l-1)}$$

Алгоритм обратного распространения ошибки

Формулы для вычисления алгоритма

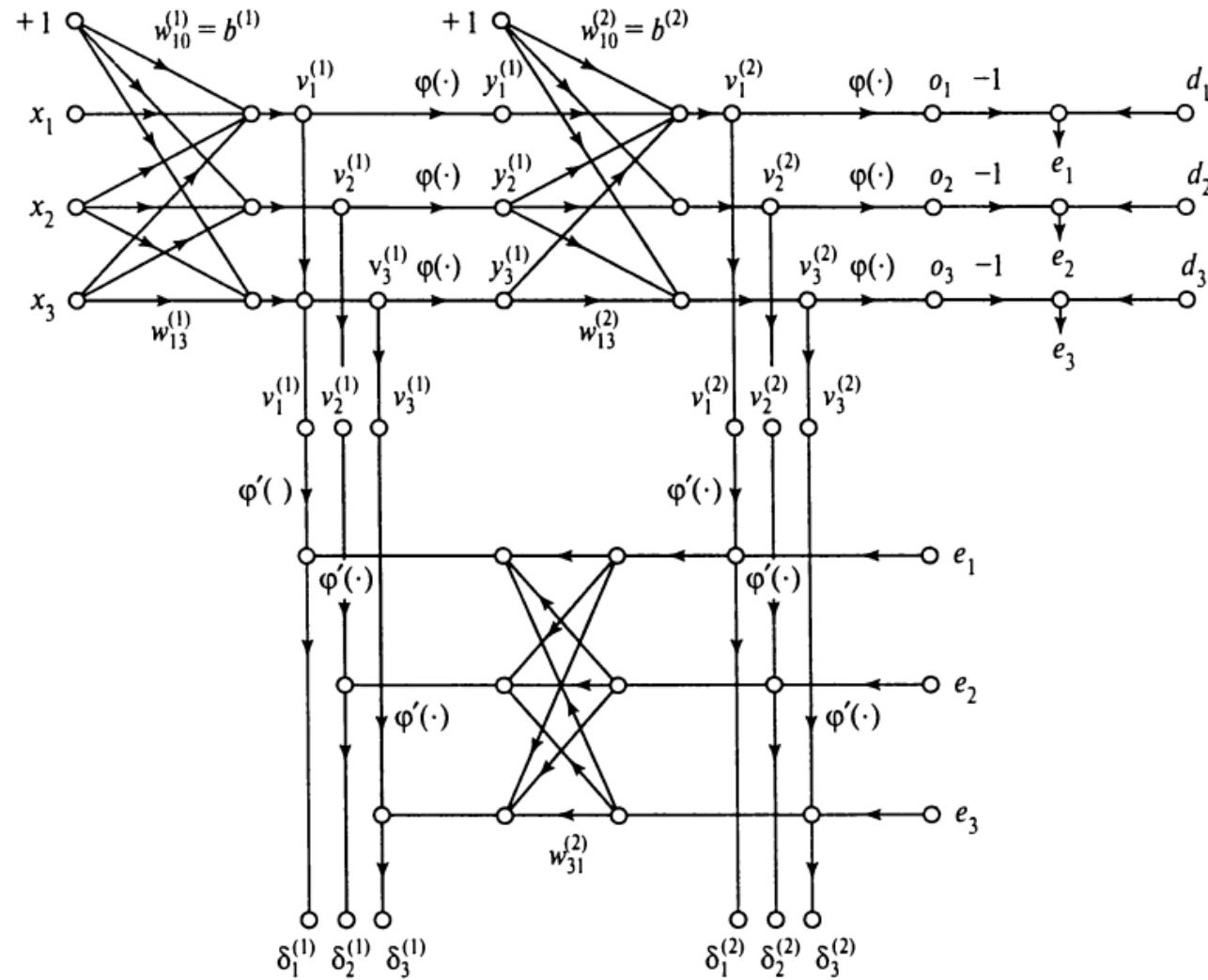
Считается, что алгоритм обратного распространения сошелся, если Евклидова норма вектора градиента достигает достаточно малых значений.

Критерием сходимости алгоритма обратного распространения является достаточно малая абсолютная интенсивность изменений среднеквадратической ошибки в течение эпохи.

Интенсивность изменения среднеквадратической ошибки обычно считается достаточно малой, если она лежит в пределах от 0,1–1% за эпоху. Иногда используется уменьшенное значение — 0,01%.

Алгоритм обратного распространения ошибки

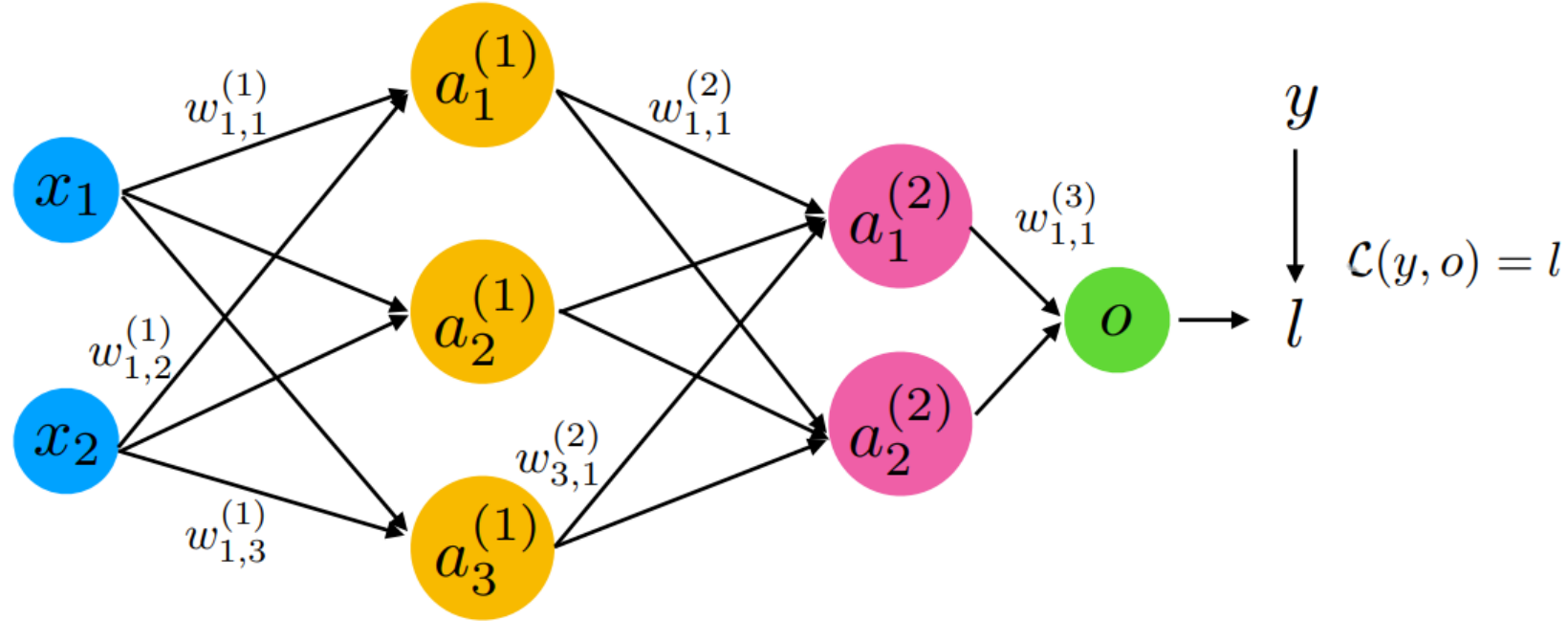
Граф передачи сигнала по методу обратного распространения ошибки



Алгоритм обратного распространения ошибки

$$\mathcal{L}(\mathbf{w}, b) = \sum_i \frac{1}{2} (\hat{y}^{[i]} - y^{[i]})^2$$

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \begin{cases} 0, & \mathbf{w}^T \mathbf{x} + b \leq 0 \\ 1, & \mathbf{w}^T \mathbf{x} + b > 0 \end{cases}$$



$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{1,1}^{(1)}} &= \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \\ &\quad + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} \end{aligned}$$

Алгоритм обратного распространения ошибки

$$\mathcal{L}(\mathbf{w}, b) = \sum_i \frac{1}{2} (\hat{y}^{[i]} - y^{[i]})^2 \qquad \hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \begin{cases} 0, & \mathbf{w}^T \mathbf{x} + b \leq 0 \\ 1, & \mathbf{w}^T \mathbf{x} + b > 0 \end{cases}$$

1. **Вход x :** Установить соответствующие значения активации $\mathbf{a}^{(1)}$ для входного слоя
2. **Прямое распространение:** Для $l = 2, \dots, L$ последовательно вычислить $\mathbf{z}^{(l)} = W^{(l)}(\mathbf{a}^{(l-1)}) + \mathbf{b}^{(l)}$ и $\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$
3. **Вычислить:** $\delta^{(L)} = \nabla_{\mathbf{a}} C \circ \sigma'(\mathbf{z}^{(L)})$
4. **Обратное распространение:** Для $l = L - 1, \dots, 2$ последовательно вычислить $\delta^{(l)} = (W^{(l+1)} \delta^{(l+1)}) \circ \sigma'(\mathbf{z}^{(l)})$
5. **Выход:** Для $l = 2, \dots, L$ вычислить

$$\nabla_{\mathbf{w}^{(l)}} C_{(x,y)} = \delta^{(l)T} \circ \mathbf{a}^{(l-1)}$$

$$\nabla_{\mathbf{b}^{(l)}} C_{(x,y)} = \delta^{(l)}$$

1. Алгоритм обратного распространения ошибки.
2. Градиентные методы.
3. Универсальные аппроксимационные свойства
Персептрона.
4. Много-классовая классификация
5. Примеры

Метод градиентного спуска

- Найти минимум функции $f(x_1, x_2): \mathbb{R}^2 \rightarrow \mathbb{R}$
- Градиент:

$$\nabla f = \left(\frac{\partial f(x_1, x_2)}{\partial x_1}; \frac{\partial f(x_1, x_2)}{\partial x_2} \right)$$

- $\Delta x := -\eta \nabla f$,
где $\eta > 0$ – малый
параметр (*скорость
обучения*)

$$x^{(0)}, x^{(1)}, \dots, x^{(n)}, x^{(n+1)} \dots$$

$$x^{(n+1)} = x^{(n)} - \eta \nabla f(x^{(n)})$$

$\eta > 0$ – *скорость обучения*

Метод градиентного спуска

Необходимо минимизировать
среднеквадратичную ошибку:

$$\mathbb{C} = \frac{1}{|V|} \sum_{(x,y) \in V} \frac{\|\alpha(x) - y\|^2}{2}$$

$$\mathbf{w} := \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathbb{C}$$

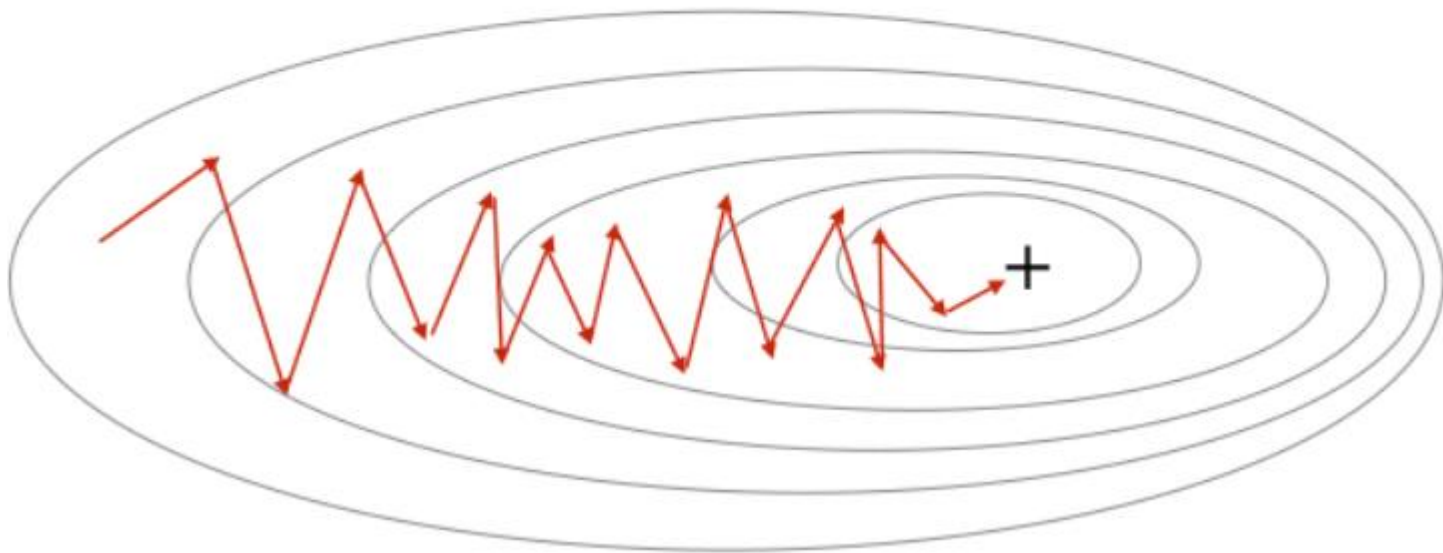
$$\mathbf{b} := \mathbf{b} - \eta \nabla_{\mathbf{b}} \mathbb{C}$$

$$\nabla_{\mathbf{w}} \mathbb{C} = \left(\frac{\partial \mathbb{C}}{\partial w_1}, \dots, \frac{\partial \mathbb{C}}{\partial w_Q} \right)$$

$$\nabla_{\mathbf{b}} \mathbb{C} = \left(\frac{\partial \mathbb{C}}{\partial b_1}, \dots, \frac{\partial \mathbb{C}}{\partial b_P} \right)$$

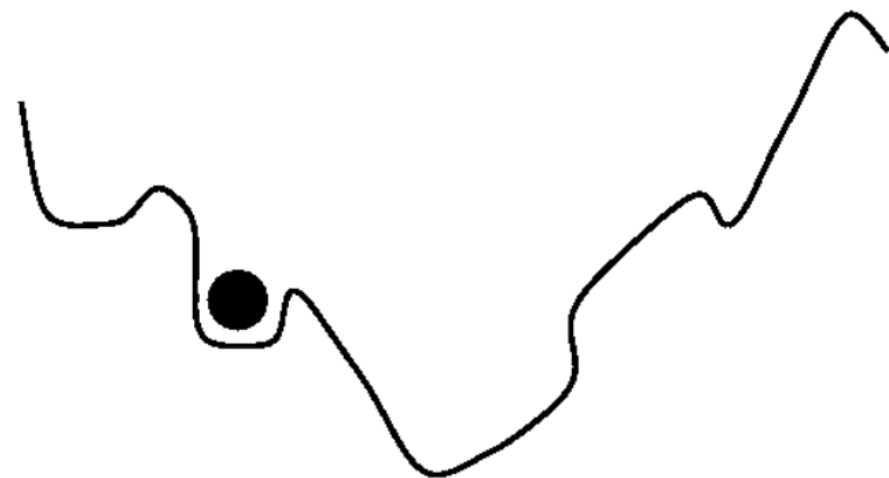
Метод стохастического градиентного спуска SGD

Stochastic Gradient Descent

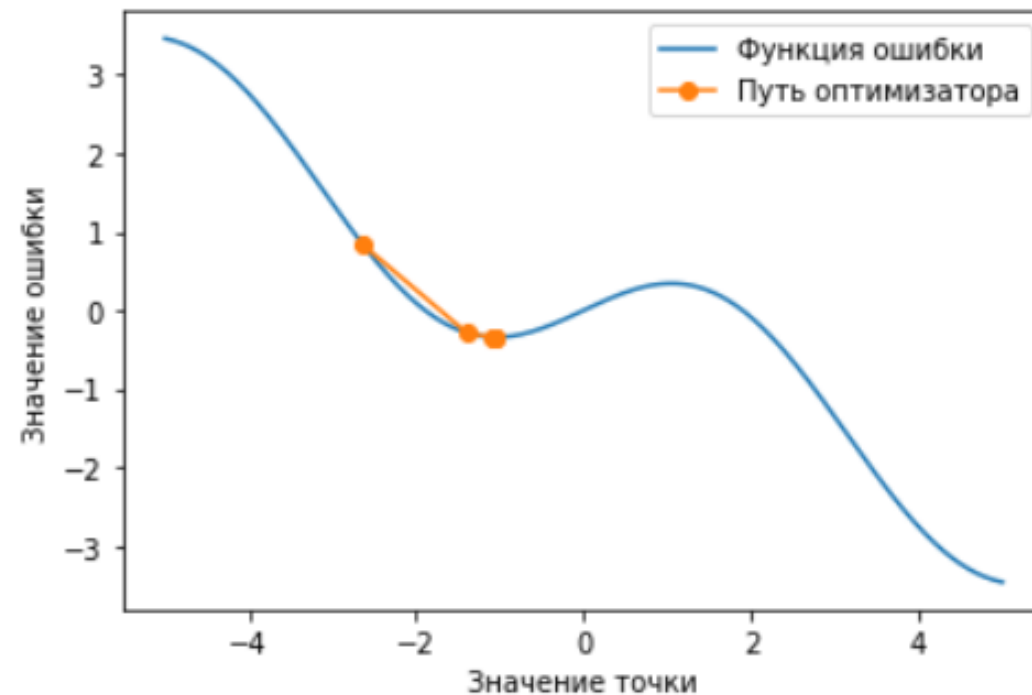
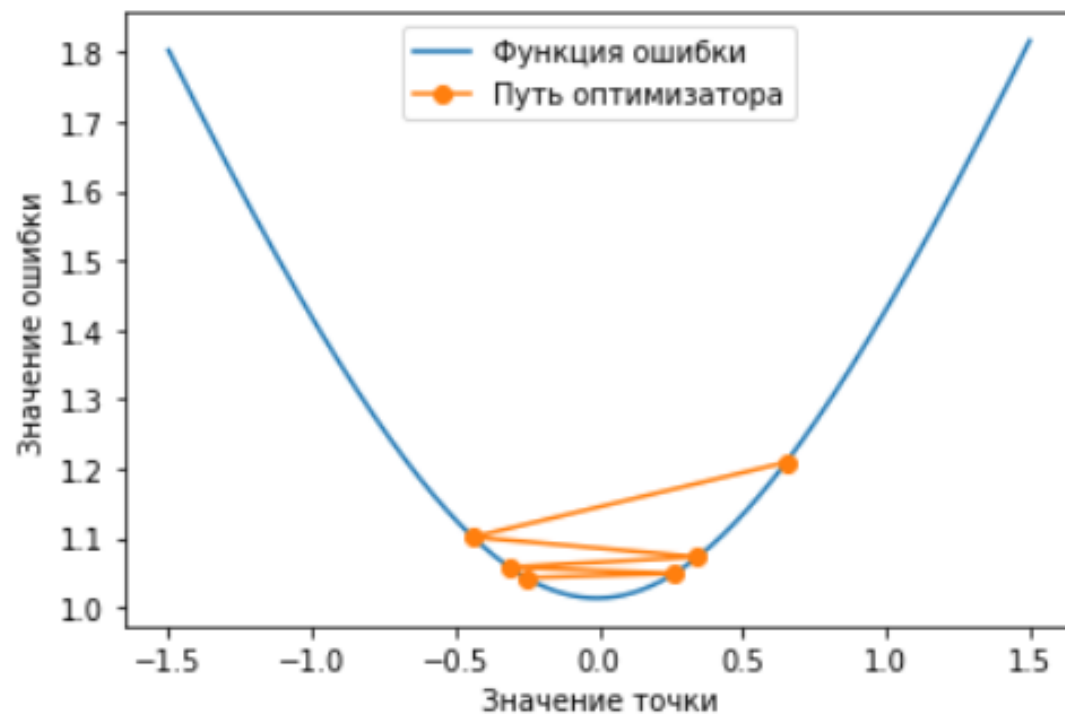


Проблемы:

- выпадение в локальные минимумы
- паралич сети
- иногда расходится
- медленная сходимость



Метод стохастического градиентного спуска SGD



Алгоритм Левенберга-Марквардта

Для достижения минимума целевой функции требуется, чтобы $\frac{dE(w(t) + p(t))}{dp(t)} = 0$. При выполнении соответствующего дифференцирования

можно получить условие оптимальности в виде:

$$g(w(t)) + H(w(t))p(t) = 0, \text{ откуда следует}$$

$$p(t) = -[H(w(t))]^{-1} g(w(t))$$

Для описания данного метода представим целевую функцию в виде:

$$E(w) = \frac{1}{2} \sum_{s=1}^M [e_s(w)]^2,$$

где $e_s = [y_s(w) - d_s(w)]$. При использовании обозначений

$$e(w) = \begin{bmatrix} e_1(w) \\ e_2(w) \\ \dots \\ e_M(w) \end{bmatrix}, \quad J(w) = \begin{bmatrix} \frac{\partial e_1}{\partial w_1} & \frac{\partial e_1}{\partial w_2} & \dots & \frac{\partial e_1}{\partial w_p} \\ \frac{\partial e_2}{\partial w_1} & \frac{\partial e_2}{\partial w_2} & \dots & \frac{\partial e_2}{\partial w_p} \\ \dots & \dots & \dots & \dots \\ \frac{\partial e_M}{\partial w_1} & \frac{\partial e_M}{\partial w_2} & \dots & \frac{\partial e_M}{\partial w_p} \end{bmatrix}, w_i = w(t).$$

Алгоритм Левенберга-Марквардта

$$G(w(t)) = [J(w(t))]^T J(w(t)) + v(t).$$

В начале процесса обучения, когда текущее решение $w(t)$ далеко от искомого решения, следует использовать значение параметра $v(t)$ намного превышающее $[J(w(t))]^T J(w(t))$. В этом случае гессиан фактически заменяется регуляризационным фактором:

$$G(w(t)) \cong v(t),$$

а направление минимизации выбирается по методу наискорейшего спуска:

$$p(t) = -\frac{g(w(t))}{v(t)}.$$

Алгоритм Левенберга-Марквардта

По мере приближения к искомому решению величина параметра $v(t)$ понижается и первое слагаемое в формуле $E(t) = \frac{v(t)^2}{r} + \sum_{i=1}^m \frac{f_i(t)^2}{r_i}$ начинает играть более важную роль. Таким образом, на эффективность алгоритма влияет правильный выбор величины $v(t)$. В методике, предложенной Д.Марквардтом, значение $v(t)$ изменяется по следующей схеме:

если $E\left(\frac{v(t-1)}{r}\right) \leq E(t)$, то принять $v(t) = \frac{v(t-1)}{r}$;

если $E\left(\frac{v(t-1)}{r}\right) > E(t)$ и $E(v(t-1)) < E(t)$, то принять $v(t) = v(t-1)$;

если $E\left(\frac{v(t-1)}{r}\right) > E(t)$ и $E(v(t-1)) > E(t)$, то увеличить m раз значение v

до достижения $E(v(t-1)r^m) \leq E(t)$, принимая $v(t) = v(t-1)r^m$,

где $E(t), v(t), E(t-1), v(t-1)$ обозначают значения целевой функции и параметра v на t -ом и $(t-1)$ -ом шагах алгоритма, а $r > 1$ - обозначает коэффициент уменьшения v .

Алгоритм Левенберга-Марквардта

Такая процедура изменения $v(t)$ применяется до момента, когда коэффициент верности отображения, рассчитываемый по формуле

$$q = \frac{E(t) - E(t-1)}{[\Delta w(t)]^T g(t) + 0,5[\Delta w(t)]^T G(t) \Delta w(t)},$$

достигнет значения, близкого к единице. При этом квадратичная аппроксимация целевой функции имеет высокую степень совпадения с истинными значениями, следовательно, регуляризационный фактор в формуле может быть приравнен нулю, а процесс определения гессиана сводится к аппроксимации первого порядка, при этом алгоритм Левенберга-Марквардта превращается в алгоритм Гаусса-Ньютона, имеющему квадратичную сходимость.

Эвристический алгоритм RPROP

Простой эвристический алгоритм, демонстрирующий высокую эффективность обучения, - это алгоритм М. Ридмиллера и Х. Брауна, называемый RPROP. В этом алгоритме при уточнении весов учитывается только знак градиентной составляющей, а ее значение игнорируется:

$$\Delta w_{ij}(t) = -\eta_{ij}(t) \operatorname{sgn} \left(\frac{\partial E(w(t))}{\partial w_{ij}} \right).$$

Коэффициент обучения подбирается индивидуально для каждого веса w_{ij} с учетом изменения значения градиента:

$$\eta_{ij}(t) = \begin{cases} \min(a\eta_{ij}(t-1), \eta_{\max}) & \text{для } S_{ij}(t)S_{ij}(t-1) > 0 \\ \max(b\eta_{ij}(t-1), \eta_{\min}) & \text{для } S_{ij}(t)S_{ij}(t-1) < 0, \\ \eta_{ij}(t-1) & \text{в остальных случаях} \end{cases}$$

где $S_{ij}(t) = \frac{\partial E(w(t))}{\partial w_{ij}}$, a и b – константы: $a=1.2$; $b=0.5$. Минимальное и

максимальное значения коэффициента обучения составляют $\eta_{\min} = 10^{-6}$ и $\eta_{\max} = 50$. Функция $\operatorname{sgn}()$ принимает значение, равное знаку градиента.

Эвристический алгоритм QuickProp

Алгоритм QuickProp содержит элементы, предотвращающие заикливание в точке неглубокого локального минимума, возникающего в результате работы нейрона на фазе насыщения сигмоидальной кривой, где из-за близости к нулю производной функции активации процесс обучения практически прекращается.

Вес w_{ij} на k -ом шаге алгоритма изменяется согласно правилу:

$$\Delta w_{ij}(k) = -\eta_k \left[\frac{\partial E(w(k))}{\partial w_{ij}} + \gamma w_{ij}(k) \right] + \alpha_{ij}^k \Delta w_{ij}(k-1)$$

Первое слагаемое соответствует оригинальному алгоритму наискорейшего спуска, последнее – фактору момента, а средний член предназначен для минимизации абсолютных значений весов. Коэффициент γ , обычно имеющий малую величину (порядка 10^{-4}), приводит к уменьшению весов вплоть до разрыва соответствующих взвешенных связей. Константа η_k – это коэффициент обучения, который в данном алгоритме может иметь ненулевое значение η_0 (как правило, $0,01 < \eta_0 < 0,6$) на старте процесса обучения, когда $\Delta w_{ij}(k-1) = 0$ или когда

$$\Delta w_{ij} \left[\frac{\partial E(w(k))}{\partial w_{ij}} + \gamma w_{ij}(k) \right] > 0, \text{ либо нулевое значение – в противном случае.}$$

Эвристический алгоритм QuickProp

Важную роль в алгоритме QuickProp играет фактор момента, который приспособливается к текущим результатам процесса обучения. Этот коэффициент подбирается индивидуально для каждого веса по правилу:

$$\alpha_{ij}^k = \begin{cases} \alpha_{\max}, & \text{где } \beta_{ij}(k) > \alpha_{\max}, \\ \beta_{ij}(k), & \text{где } S_{ij}(k) \Delta w_{ij}(k-1) \beta_{ij}(k) < 0; \end{cases}$$

причем:

$$S_{ij}(k) = \frac{\partial E(w(k))}{\partial w_{ij}} + \gamma w_{ij}(k)$$

$$\beta_{ij}(k) = \frac{S_{ij}(k)}{S_{ij}(k-1) - S_{ij}(k)}$$

Константа $\alpha_{\max}$ - это максимальное значение коэффициента момента, которое принимается равным 1,75.

Также известна упрощенная версия алгоритма QuickProp, в которой значения весов изменяются в соответствии с правилом:

$$\Delta w_{ij}^k = \begin{cases} \alpha_{ij}(k) \Delta w_{ij}(k-1) & \text{для } \Delta w_{ij}(k-1) \neq 0 \\ \eta_0(k) \frac{\partial E}{\partial w_{ij}} & \end{cases}$$

$$\alpha_{ij}(k) = \min \left(\frac{S_{ij}(k)}{S_{ij}(k) - S_{ij}(k-1)}, a_{\max} \right)$$

$$\text{где } S_{ij}(k) = \frac{\partial E(w(k))}{\partial w_{ij}}.$$

Универсальные аппроксимационные свойства персептрона

Теорема Колмогорова

Теорема Колмогорова. *Каждая непрерывная функция n переменных, заданная на единичном кубе n -мерного пространства, представима в виде*

$$f(x_1, x_2, \dots, x_n) = \sum_{q=1}^{2n+1} h_q \left[\sum_{p=1}^n \varphi_q^p(x_p) \right], \quad (1)$$

где функции $h_q(u)$ непрерывны, а $\varphi_q^p(x_p)$, кроме того, еще и стандартны, т.е. не зависят от выбора функции f .

В частности, каждая непрерывная функция двух переменных x, y представима в виде

$$f(x, y) = \sum_{q=1}^5 [\varphi_q(x) + \psi_q(y)]. \quad (2)$$

Теорема Стоуна

Любой многочлен от многих переменных может быть получен из одного произвольного нелинейного многочлена от одного переменного с помощью линейных операций и суперпозиции.

По т. Вейерштрасса непрерывную функцию нескольких переменных $f(x_1, \dots, x_n)$ на замкнутом ограниченном множестве Q можно равномерно приблизить последовательностью полиномов.

*Чтобы обобщить и усилить теорему Вейерштрасса, рассмотрим компактное пространство X и алгебру $C(X)$ непрерывных функций на X с вещественными значениями. Сильным обобщением этой теоремы является **теорема Стоуна**.*

*Пусть $E \subseteq C(X)$ - замкнутая подалгебра в $C(X)$, $1 \in E$ и функции из E разделяют точки в X ,
т.е. для любых различных $x, y \in X$ существует такая функция $g \in E$, что $g(x) \neq g(y)$.*

Тогда $E = C(X)$.

Теорема Стоуна

Теорема Стоуна дает модель конструирования: достаточно взять произвольный набор функций, разделяющих точки, построить кольцо многочленов от них - в результате получится плотное в $C(X)$ множество функций.

Чтобы получить требуемое обобщение, перейдем от рассмотрения колец функций к изучению их алгебр, замкнутых относительно некоторой нелинейной унарной операции.

Определение. Пусть $E \subseteq C(X)$ – линейное пространство, $C(R)$ –пространство непрерывных функций на действительной оси R ,
 $f \in C(R)$ нелинейная функций и для любого $g \in E$ выполнено условие $f(g) \in E$,

Тогда пространство E замкнуто относительно унарной операции f .

Пусть $E \subseteq C(X)$ – замкнутое линейное подпространство в $C(X)$, $1 \in E$, функции из E разделяют точки в X и E замкнуто относительно нелинейной унарной операции $f(x) = x^2$. Тогда $E = C(X)$.

Универсальные аппроксимационные свойства персептрона

Теорема

Теорема 1. Пусть $E \subseteq C(X)$ – замкнутое линейное подпространство в $C(X)$, $1 \in E$, функции из E разделяют точки в X и E замкнуто относительно нелинейной унарной операции $f \in C(\mathbf{R})$. Тогда $E = C(X)$.

Доказательство. Рассмотрим множество всех непрерывных унарных операций указанного вида, относительно которых E инвариантно. Обозначим через P_E множество всех таких непрерывных функций $p \in C(\mathbf{R})$, что $p(E) \subseteq E$, т.е. для любого $g \in E$ выполнено $p(g) \in E$. Множество P_E обладает следующими свойствами:

- 1) P_E – полугруппа относительно суперпозиции функций;
- 2) P_E – замкнутое линейное подпространство в $C(\mathbf{R})$ (в топологии равномерной сходимости на компактах);
- 3) $1 \in P_E$ и $\text{id} \in P_E$ ($\text{id}(x) \equiv x$);
- 4) P_E включает хотя бы одну непрерывную нелинейную функцию.

Универсальные аппроксимационные свойства персептрона

Теорема 2. Пусть множество $P \subseteq C(\mathbf{R})$ – удовлетворяет условиям 1-4.

Тогда $P = C(\mathbf{R})$.

Доказательство опирается на 3 леммы.

Обобщение Теоремы Стоуна

Лемма 1. В условиях теоремы 2 существует дважды непрерывно дифференцируемая функция $g \in P$, не являющаяся линейной.

Доказательство. Пусть $v(x) \in C^\infty(\mathbf{R})$, $v(x) = 0$ при $|x| > 1$, $\int_{\mathbf{R}} v(x) dx = 1$. Рассмотрим оператор осреднения

$$J_\varepsilon f(x) = \int_{\mathbf{R}} f(x+y) \frac{1}{\varepsilon} v\left(\frac{y}{\varepsilon}\right) dy.$$

Для любого $\varepsilon > 0$ выполнено $J_\varepsilon f(x) \in P$. Действительно, $f(x+y) \in E$ для каждого фиксированного y (поскольку константы принадлежат E и E замкнуто относительно линейных операций и суперпозиции функций). Интеграл $J_\varepsilon f(x)$ принадлежит E , так как E является замкнутым линейным подпространством в $C(\mathbf{R})$, а этот интеграл – предел конечных сумм.

Функция $J_\varepsilon f(x)$ принадлежит $C^\infty(\mathbf{R})$, так как

$$J_\varepsilon f(x) = \int_{\mathbf{R}} f(x+y) \frac{1}{\varepsilon} v\left(\frac{y}{\varepsilon}\right) dy = \int_{\mathbf{R}} f(z) \frac{1}{\varepsilon} v\left(\frac{z-x}{\varepsilon}\right) dz.$$

Существует такое $\varepsilon > 0$, что функция $g = J_\varepsilon f$ не является линейной, поскольку $J_\varepsilon f \rightarrow f$ при $\varepsilon \rightarrow 0$, пространство линейных функций замкнуто, а f не является линейной функцией. Таким образом, в предположениях леммы существует нелинейная функция $g \in P \cap C^\infty(\mathbf{R})$, которую можно выбрать в виде $g = J_\varepsilon f$. $\square$

Обобщение Теоремы Стоуна

Лемма 2. Пусть в условиях теоремы 2 существует дважды непрерывно дифференцируемая функция $g \in P$, не являющаяся линейной. Тогда функция $q(x) = x^2$ принадлежит P .

Доказательство. Существует точка x_0 , для которой $g''(x_0) \neq 0$. Обозначим $r(x) = 2(g(x + x_0) - g(x_0) - xg'(x_0))/g''(x_0)$. Очевидно, что $r \in P$, $r(0) = 0$, $r'(0) = 0$, $r''(0) = 2$, $r(x) = x^2 + o(x^2)$. Поэтому $r(\varepsilon x)/\varepsilon^2 \rightarrow x^2$ при $\varepsilon \rightarrow 0$. Поскольку P замкнуто, то функция $q(x) = x^2$ принадлежит P . $\square$

Доказательство теоремы

Лемма 3. Пусть в условиях теоремы 2 функция $q(x) = x^2$ принадлежит P . Тогда P является кольцом и для любых $f, g \in P$ их произведение fg принадлежит P .

Доказательство. Действительно, $fg = 1/2 [(f + g)^2 - f^2 - g^2]$, и так как P замкнуто относительно суперпозиции и линейных операций, то $fg \in P$. $\square$

Доказательство теоремы 2 заканчивается использованием классической теоремы Вейерштрасса о приближении функций многочленами: из лемм 1–3 следует, что в условиях теоремы 2 множество P является кольцом и, в частности, содержит все многочлены (которые получаются из 1 и id с помощью умножения и линейных операций). Отсюда по теореме Вейерштрасса следует, что $P = C(\mathbf{R})$.

Теоремы 1, 2 можно трактовать как утверждения об универсальных аппроксимационных свойствах любой нелинейности: с помощью линейных операций и каскадного соединения можно из произвольных нелинейных элементов получить любой требуемый результат с любой наперед заданной точностью.

1. Алгоритм обратного распространения ошибки.
2. Градиентные методы.
3. Универсальные аппроксимационные свойства Персептрона.
4. **Много-классовая классификация**
5. Примеры

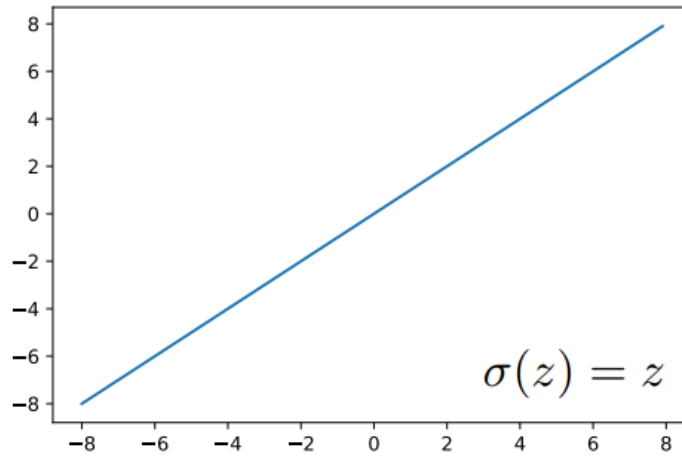
Много-классовая классификация (Multinomial Logistic Regression / Softmax Regression)

Onehot Encoding

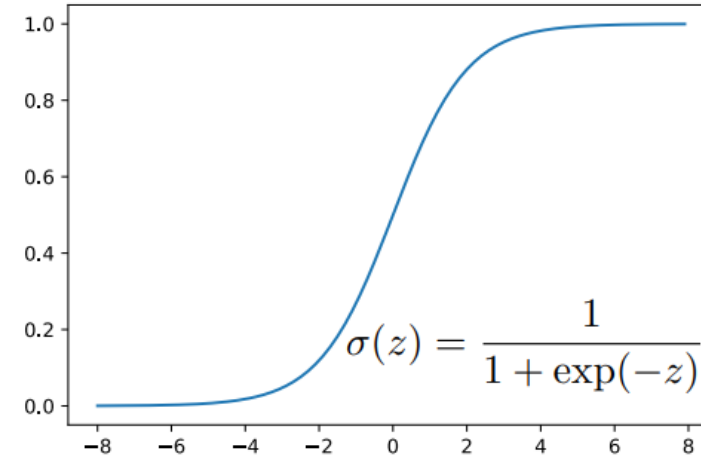
class labels		class_0	class_1	class_2	class_3
0	→	1	0	0	0
1		0	1	0	0
3		0	0	0	1
2		0	0	1	0

Выбор активационной функции

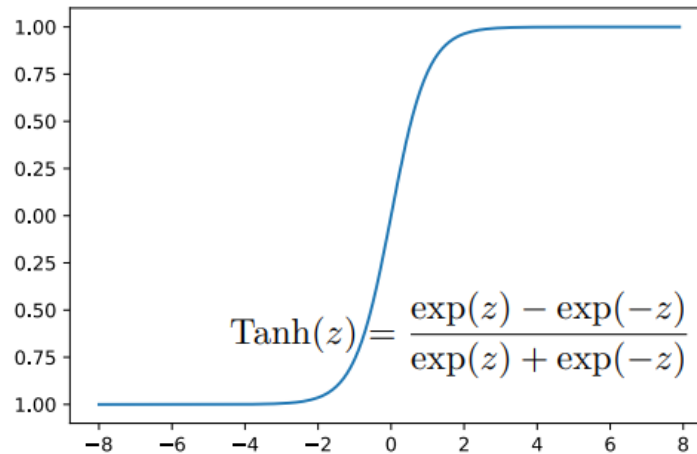
Identity



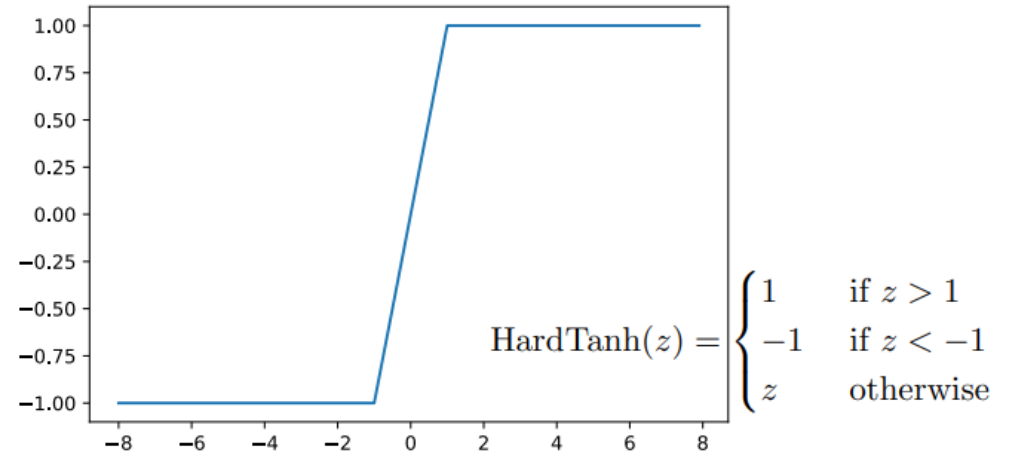
(Logistic) Sigmoid



Tanh ("tanH")

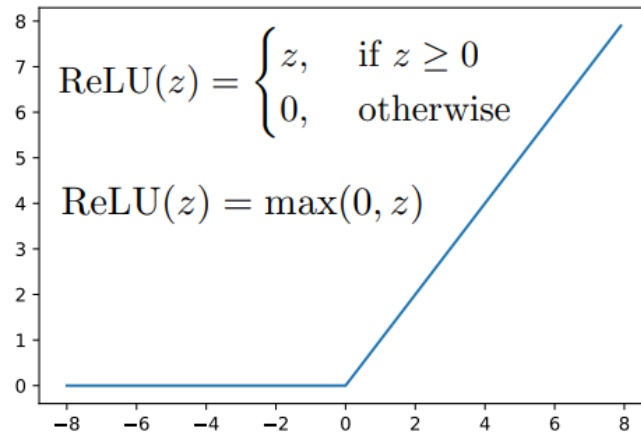


Hard Tanh

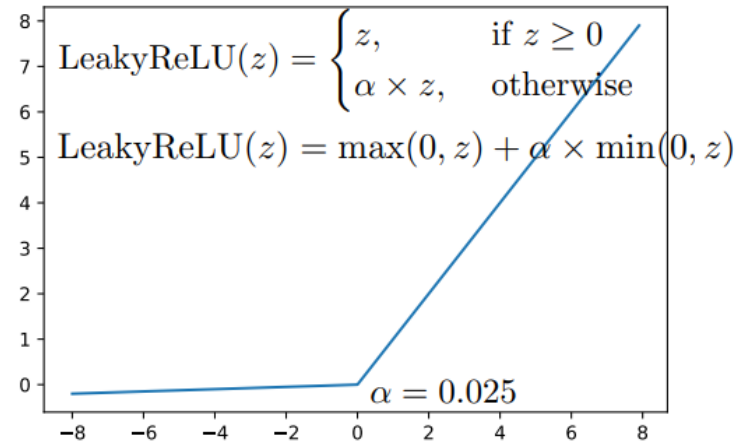


Выбор активационной функции

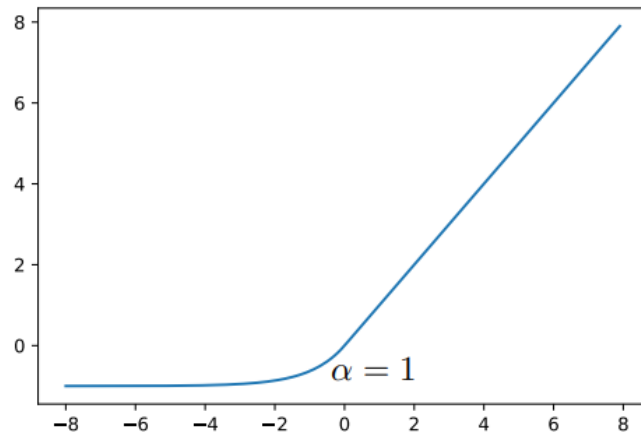
ReLU (Rectified Linear Unit)



Leaky ReLU



ELU (Exponential Linear Unit)



PReLU (Parameterized Rectified Linear Unit)

$$\text{PReLU}(z) = \begin{cases} z, & \text{if } z \geq 0 \\ \alpha z, & \text{otherwise} \end{cases}$$

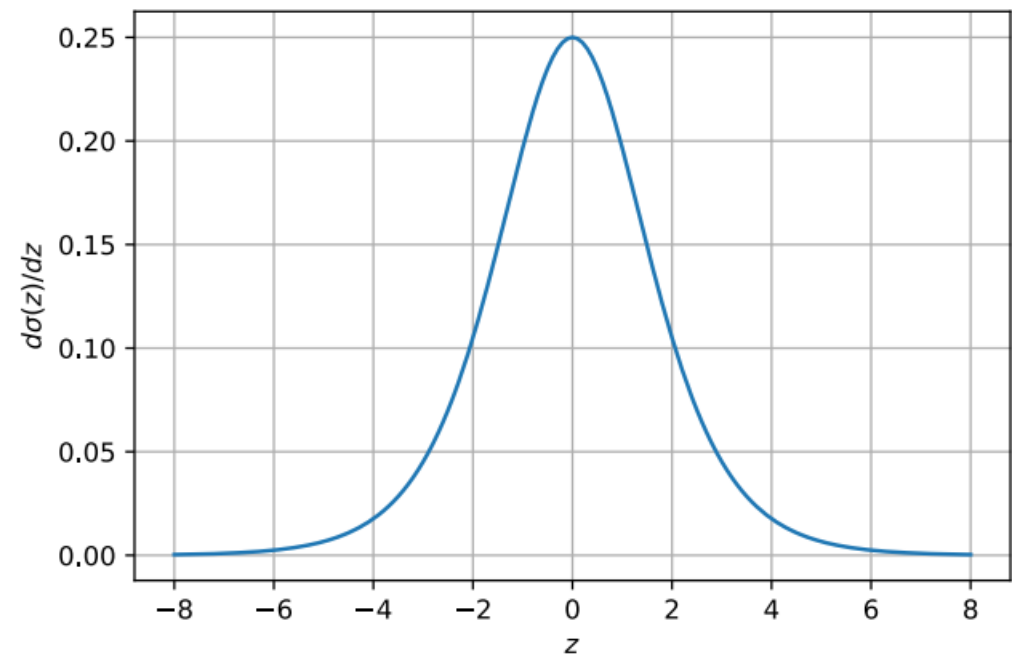
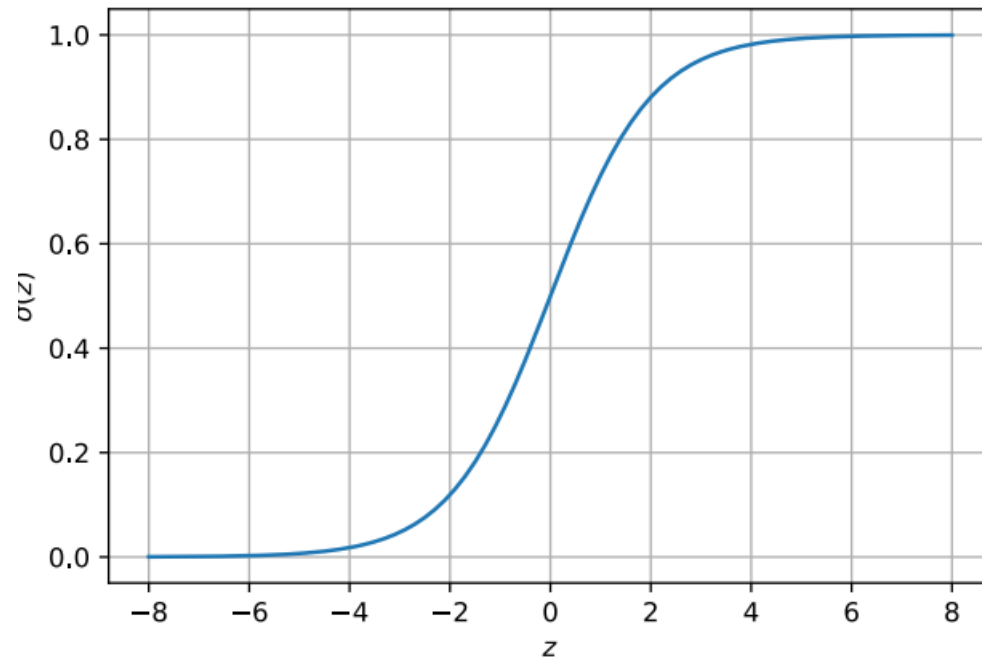
$$\text{PReLU}(z) = \max(0, z) + \alpha \times \min(0, z)$$

Пример. Бинарная классификация (ошибка MSE)

Активационная функция – логистический сигмоид)

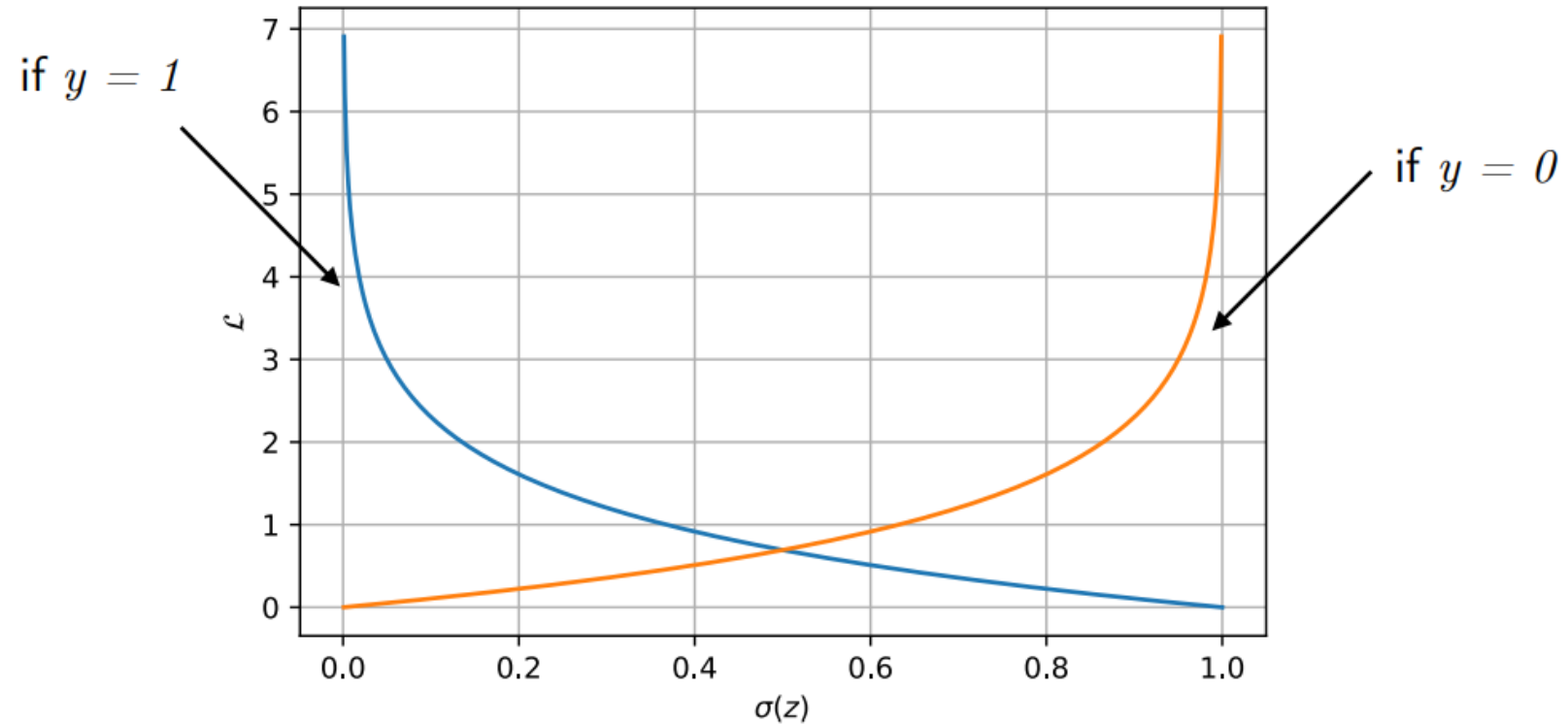
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



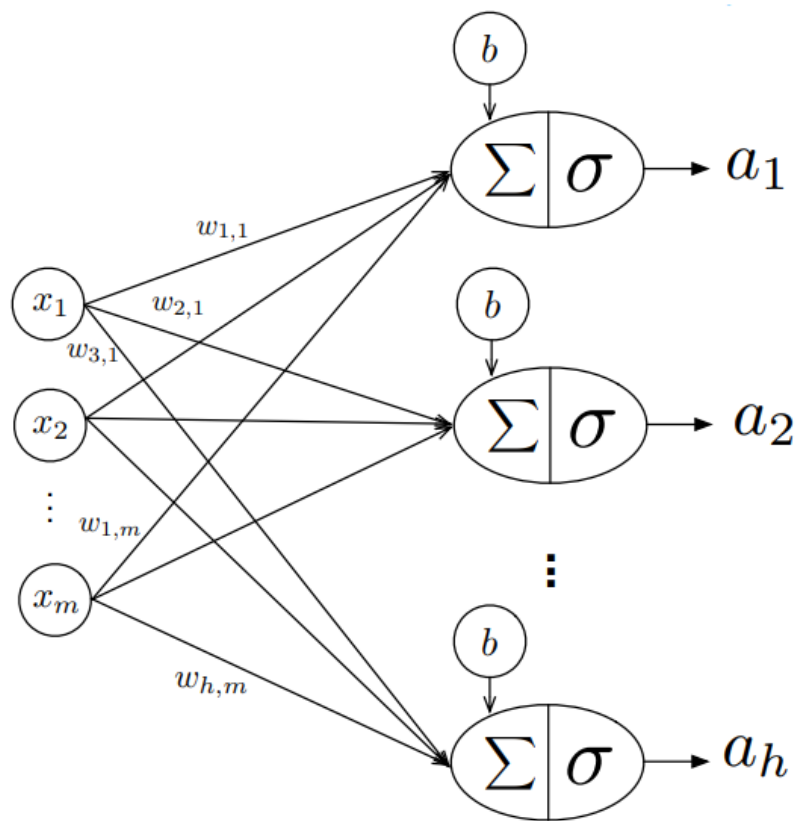
Пример. Бинарная классификация

Ошибка после 1 шага обучения



$$\mathcal{L}(\mathbf{w}) = -y^{(i)} \log(\sigma(z^{(i)})) + (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))$$

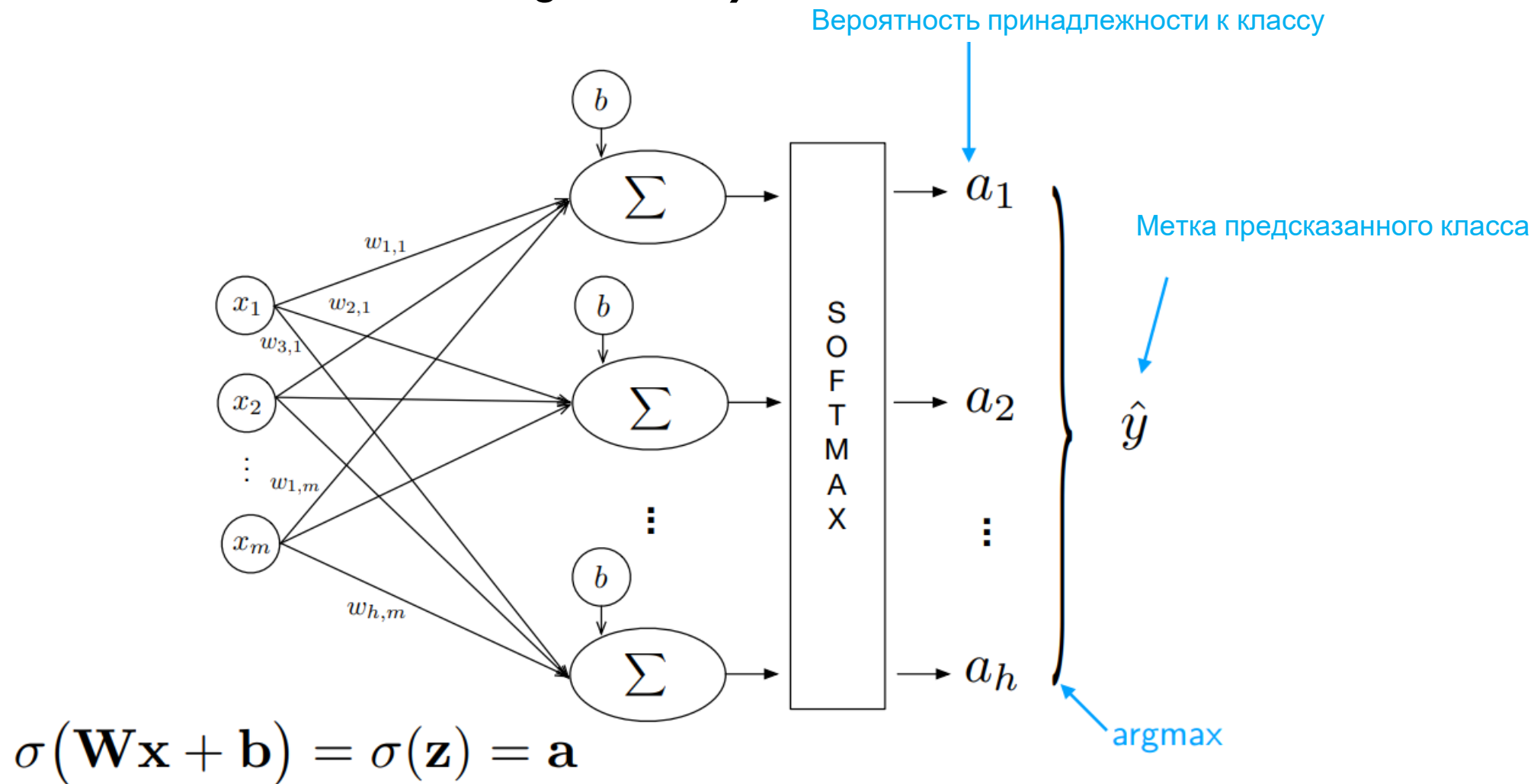
Много-классовая классификация (k-количество классов)



$$\sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) = \sigma(\mathbf{z}) = \mathbf{a}$$

$$\mathbf{W} \in \mathbb{R}^{k \times m}$$

Много-классовая классификация (Multinomial Logistic Regression / Softmax Regression)



Много-классовая классификация

Активационная функция Softmax

Сумма всех выходных сигналов равна 1

Выходной сигнал j -того нейрона интерпретируется как вероятность того, что правильный ответ есть j

$$P(y = t \mid z_t^{(i)}) = \sigma_{\text{softmax}}(z_t^{(i)}) = \frac{e^{z_t^{(i)}}}{\sum_{j=1}^k e^{z_j^{(i)}}}$$

$$t \in \{j \dots k\}$$

 (k-количество классов)

Много-классовая классификация

Активационная функция Softmax. Свойства.

$$1) 0 < \frac{e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} < 1 \Rightarrow 0 < a_j^{(L)} < 1$$

$$2) \sum_{j=1}^n a_j^{(L)} = \sum_{j=1}^n \frac{e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} = \frac{\sum_{j=1}^n e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} = 1$$

Много-классовая классификация

Функция потерь

Loss function

$$\mathcal{L}_{\text{binary}} = - \sum_{i=1}^n \left(y^{[i]} \log(a^{[i]}) + (1 - y^{[i]}) \log(1 - a^{[i]}) \right)$$

one-hot encoded labels

$$\mathcal{L}_{\text{multiclass}} = \sum_{i=1}^n \sum_{j=1}^k -y_j^{[i]} \log(a_k^{[i]})$$

Cross Entropy

Много-классовая классификация

Функция потерь. Свойства

$$C = - \sum_j y_j \ln y_j$$

1) $C \geq 0$

2) Минимум C достигается при $\mathbf{a}^{(L)} = \mathbf{y}$:

$$\min_{\mathbf{a}^{(L)}} C = - \sum_j y_j \ln y_j$$

Много-классовая классификация

Функция потерь. Свойства

1) $C \geq 0$

$$\left. \begin{array}{l} y_j \geq 0 \\ 0 < a_j^{(L)} < 1 \Rightarrow \ln a_j^{(L)} < 0 \end{array} \right\} \Rightarrow -y_j \ln a_j^{(L)} \geq 0 \Rightarrow$$

$$C = - \sum_j y_j \ln a_j^{(L)} \geq 0$$

$$\frac{e^{z_t^{(i)}}}{\sum_{j=1}^k e^{z_j^{(i)}}} < 1$$

Много-классовая классификация

Функция потерь. Влияние нейрона

$$\begin{aligned}
 \delta_i^{(L)} &= \frac{\partial C}{\partial z_i^{(L)}} = \sum_{j=1}^n \frac{\partial C}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_i^{(L)}} = \sum_{j=1}^n -\frac{y_j}{a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_i^{(L)}} = -\sum_{j=1}^n \frac{y_j}{a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_i^{(L)}} \\
 &= -\frac{y_i}{a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} - \sum_{\substack{j=1 \\ j \neq i}}^n \frac{y_j}{a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_i^{(L)}} = -\frac{y_i}{a_i^{(L)}} a_i^{(L)} (1 - a_i^{(L)}) - \sum_{\substack{j=1 \\ j \neq i}}^n -\frac{y_j}{a_j^{(L)}} \cdot a_i^{(L)} a_j^{(L)} \\
 &= -y_i + y_i a_i^{(L)} + \sum_{\substack{j=1 \\ j \neq i}}^n y_j a_i^{(L)} = -y_i + \sum_{j=1}^n y_j a_i^{(L)} = -y_i + a_i^{(L)} \sum_{j=1}^n y_j \\
 &= -y_i + a_i^{(L)} = a_i^{(L)} - y_i
 \end{aligned}$$

$$\frac{\partial C}{\partial a_j^{(L)}} = -\frac{\partial}{\partial a_j^{(L)}} \left(\sum_{i=1}^n y_i \ln a_i^{(L)} \right) = -\frac{y_j}{a_j^{(L)}}$$

$$\begin{aligned}
 \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} &= \frac{\partial}{\partial z_j^{(L)}} \left(\frac{e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} \right) = \frac{e^{z_j^{(L)}} \sum_{i=1}^n e^{z_i^{(L)}} - e^{2z_j^{(L)}}}{\left(\sum_{i=1}^n e^{z_i^{(L)}} \right)^2} = \\
 &= \frac{e^{z_j^{(L)}} \sum_{i=1}^n e^{z_i^{(L)}} - e^{2z_j^{(L)}}}{\left(\sum_{i=1}^n e^{z_i^{(L)}} \right)^2} = \frac{e^{z_j^{(L)}} \sum_{i=1}^n e^{z_i^{(L)}}}{\left(\sum_{i=1}^n e^{z_i^{(L)}} \right)^2} - \frac{e^{2z_j^{(L)}}}{\left(\sum_{i=1}^n e^{z_i^{(L)}} \right)^2} = \\
 &= a_j^{(L)} - (a_j^{(L)})^2 = a_j^{(L)} (1 - a_j^{(L)})
 \end{aligned}$$

$$\begin{aligned}
 \frac{\partial a_j^{(L)}}{\partial z_i^{(L)}} &= \frac{\partial}{\partial z_i^{(L)}} \left(\frac{e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} \right) = \frac{-e^{z_i^{(L)}} e^{z_j^{(L)}}}{\left(\sum_{i=1}^n e^{z_i^{(L)}} \right)^2} = \\
 &= \frac{-e^{z_i^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} \cdot \frac{e^{z_j^{(L)}}}{\sum_{i=1}^n e^{z_i^{(L)}}} = -a_i^{(L)} a_j^{(L)}
 \end{aligned}$$

Много-классовая классификация

Функция потерь. Влияние нейрона

$$\frac{\partial C}{\partial b_i^{(L)}} = a_i^{(L)} - y_i$$

$$\frac{\partial C}{\partial b_i^{(L)}} = \frac{\partial C}{\partial z_i^{(L)}} \cdot \frac{\partial z_i^{(L)}}{\partial b_i^{(L)}} = \frac{\partial C}{\partial z_i^{(L)}} \cdot 1 = \frac{\partial C}{\partial z_i^{(L)}} = a_i^{(L)} - y_i$$

$$\frac{\partial C}{\partial w_{ji}^{(L)}} = a_j^{(L-1)} (a_i^{(L)} - y_i)$$

$$\frac{\partial C}{\partial w_{ji}^{(L)}} = \frac{\partial C}{\partial z_i^{(L)}} \cdot \frac{\partial z_i^{(L)}}{\partial w_{ji}^{(L)}} = \frac{\partial C}{\partial z_i^{(L)}} \cdot a_j^{(L-1)} = a_j^{(L-1)} \frac{\partial C}{\partial z_i^{(L)}} = a_j^{(L-1)} (a_i^{(L)} - y_i)$$

$$z_i^{(L)} = b_i^{(L)} + \sum_j \left[w_{ji}^{(L)} \right] a_j^{(L-1)}$$

Много-классовая классификация

Функция потерь. Влияние нейрона

$$\mathbb{C} = -\frac{1}{|V|} \sum_{(x,y) \in V} \sum_j y_j \ln a_j^{(L)}$$

$$\frac{\partial \mathcal{C}}{\partial b_i^{(L)}} = a_i^{(L)} - y_i$$

$$\frac{\partial \mathcal{C}}{\partial w_{ji}^{(L)}} = a_j^{(L-1)} (a_i^{(L)} - y_i)$$

Много-классовая классификация

Активационная функция Softmax

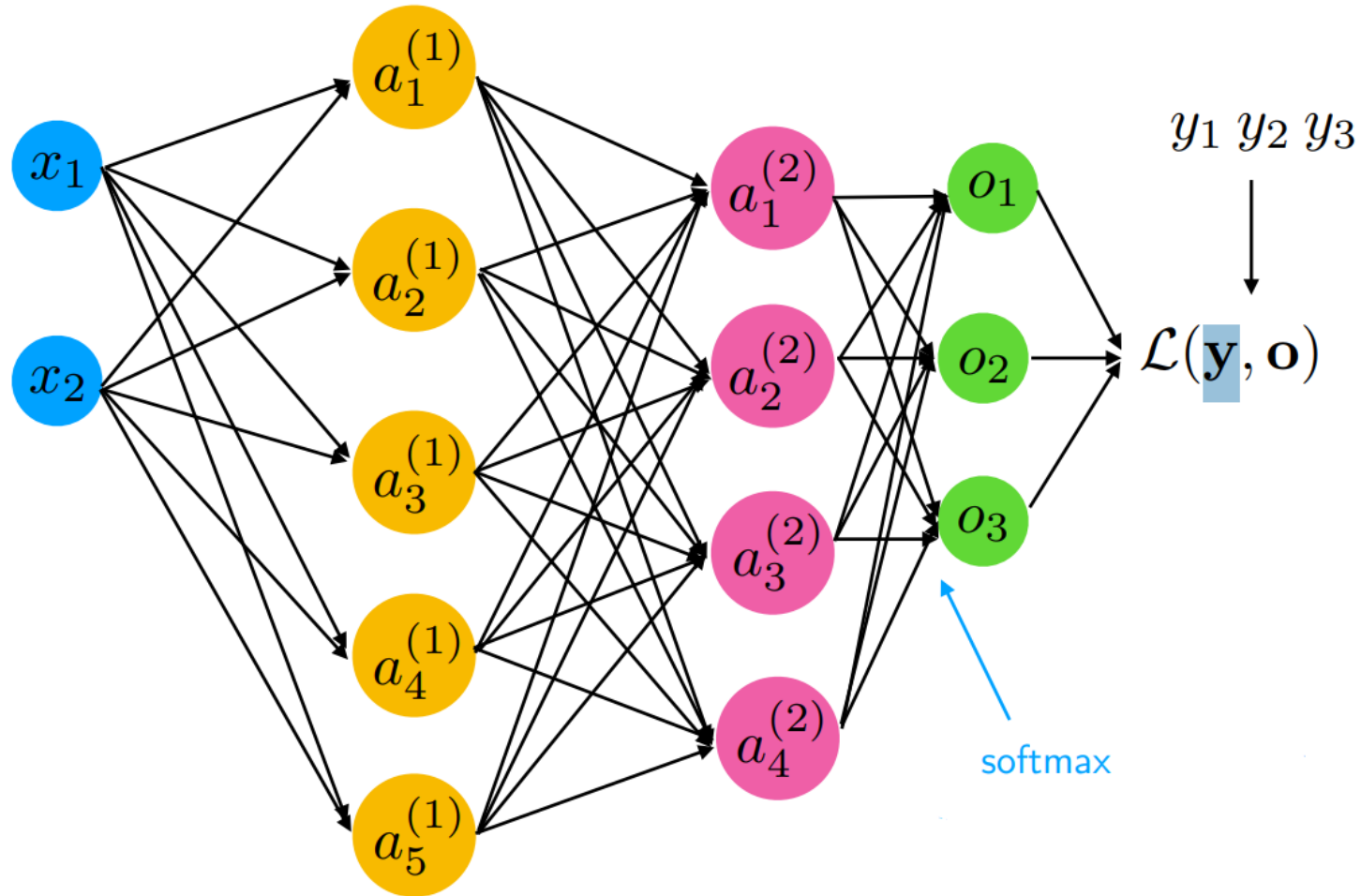
$$P(y = t \mid z_t^{(i)}) = \sigma_{\text{softmax}}(z_t^{(i)}) = \frac{e^{z_t^{(i)}}}{\sum_{j=1}^k e^{z_j^{(i)}}}$$

$$t \in \{j \dots k\}$$



(k-количество классов)

Много-классовая классификация



$$\mathcal{L}_{\text{multiclass}} = \sum_{i=1}^n \sum_{j=1}^k -y_j^{[i]} \log(a_k^{[i]})$$

Cross Entropy

Много-классовая классификация

Пример. Loss function Cross Entropy

(3 - количество классов)

$$\mathbf{Y}_{\text{onehot}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{A}_{\text{softmax outputs}} = \begin{bmatrix} 0.3792 & 0.3104 & 0.3104 \\ 0.3072 & 0.4147 & 0.2780 \\ 0.4263 & 0.2248 & 0.3490 \\ 0.2668 & 0.2978 & 0.4354 \end{bmatrix}$$

$$\mathcal{L}_{\text{multiclass}} = \sum_{i=1}^n \sum_{j=1}^k -y_j^{[i]} \log(a_k^{[i]})$$

$$\begin{aligned} \mathcal{L}^{[1]} &= [(-1) \cdot \log(0.3792)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &= 0.969692... \end{aligned}$$

Много-классовая классификация

Пример. Loss function Cross Entropy

(3 - количество классов)

$$\mathbf{Y}_{\text{onehot}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{A}_{\text{softmax outputs}} = \begin{bmatrix} 0.3792 & 0.3104 & 0.3104 \\ 0.3072 & 0.4147 & 0.2780 \\ 0.4263 & 0.2248 & 0.3490 \\ 0.2668 & 0.2978 & 0.4354 \end{bmatrix}$$

$$\begin{aligned} \mathcal{L}^{[1]} &= [(-1) \cdot \log(0.3792)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &= 0.969692... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[2]} &= [(-0) \cdot \log(0.3072)] \\ &\quad + [(-1) \cdot \log(0.4147)] \\ &\quad + [(-0) \cdot \log(0.2780)] \\ &= 0.880200... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[3]} &= [(-0) \cdot \log(0.4263)] \\ &\quad + [(-0) \cdot \log(0.2248)] \\ &\quad + [(-1) \cdot \log(0.3490)] \\ &= 1.05268... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[4]} &= [(-0) \cdot \log(0.2668)] \\ &\quad + [(-0) \cdot \log(0.2978)] \\ &\quad + [(-1) \cdot \log(0.4354)] \\ &= 0.831490... \end{aligned}$$

Много-классовая классификация

Пример. Loss function Cross Entropy

(3 - количество классов)

$$\mathbf{Y}_{\text{onehot}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{A}_{\text{softmax outputs}} = \begin{bmatrix} 0.3792 & 0.3104 & 0.3104 \\ 0.3072 & 0.4147 & 0.2780 \\ 0.4263 & 0.2248 & 0.3490 \\ 0.2668 & 0.2978 & 0.4354 \end{bmatrix}$$

$$\begin{aligned} \mathcal{L}^{[1]} &= [(-1) \cdot \log(0.3792)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &= 0.969692... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[2]} &= [(-0) \cdot \log(0.3072)] \\ &\quad + [(-1) \cdot \log(0.4147)] \\ &\quad + [(-0) \cdot \log(0.2780)] \\ &= 0.880200... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[3]} &= [(-0) \cdot \log(0.4263)] \\ &\quad + [(-0) \cdot \log(0.2248)] \\ &\quad + [(-1) \cdot \log(0.3490)] \\ &= 1.05268... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[4]} &= [(-0) \cdot \log(0.2668)] \\ &\quad + [(-0) \cdot \log(0.2978)] \\ &\quad + [(-1) \cdot \log(0.4354)] \\ &= 0.831490... \end{aligned}$$

$$\mathcal{L}_{\text{multiclass}} = \sum_{i=1}^n \sum_{j=1}^k -y_j^{[i]} \log(a_k^{[i]})$$
$$\approx 0.9335$$

Много-классовая классификация

Пример. Loss function Cross Entropy

(3 - количество классов)

$$\mathbf{Y}_{\text{onehot}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{A}_{\text{softmax outputs}} = \begin{bmatrix} 0.3792 & 0.3104 & 0.3104 \\ 0.3072 & 0.4147 & 0.2780 \\ 0.4263 & 0.2248 & 0.3490 \\ 0.2668 & 0.2978 & 0.4354 \end{bmatrix}$$

$$\begin{aligned} \mathcal{L}^{[1]} &= [(-1) \cdot \log(0.3792)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &\quad + [(-0) \cdot \log(0.3104)] \\ &= 0.969692... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[2]} &= [(-0) \cdot \log(0.3072)] \\ &\quad + [(-1) \cdot \log(0.4147)] \\ &\quad + [(-0) \cdot \log(0.2780)] \\ &= 0.880200... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[3]} &= [(-0) \cdot \log(0.4263)] \\ &\quad + [(-0) \cdot \log(0.2248)] \\ &\quad + [(-1) \cdot \log(0.3490)] \\ &= 1.05268... \end{aligned}$$

$$\begin{aligned} \mathcal{L}^{[4]} &= [(-0) \cdot \log(0.2668)] \\ &\quad + [(-0) \cdot \log(0.2978)] \\ &\quad + [(-1) \cdot \log(0.4354)] \\ &= 0.831490... \end{aligned}$$

$$\mathcal{L}_{\text{multiclass}} = \sum_{i=1}^n \sum_{j=1}^k -y_j^{[i]} \log(a_k^{[i]})$$
$$\approx 0.9335$$